

CSC9008 Cyber-Physical Systems

The good plate: Final Presentation

Course Instructor: Prof. Chao Wang

Students: Xin-Shao Hon, Zhen-Rong Wu, Darrin Lin

Department of Computer Science and Information Engineering
National Taiwan Normal University

June. 2, 2026

Agenda

- 1 System Architecture
- 2 Recall: Basic background
- 3 Complementary Filter for Attitude Estimation
- 4 PID Control System
- 5 String-Length Geometry for MG90S Servos
- 6 System analysis
- 7 Live Demo

Project Overview

Goal

Build a self-stabilizing platform that keeps objects level despite external disturbances (manual tilting, vibration, earthquakes).

Hardware:

- STM32F401CCU6 (MCU)
- GY-521 / MPU6050 (IMU)
- 4 × MG90S servos
- Pulleys + strings
- Central universal joint

Control flow:

- Sense tilt via IMU
- Compute attitude (pitch, roll)
- PID computes correction
- Servos pull strings
- Platform tilts back to level

Hardware Price

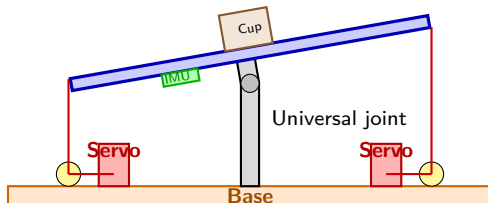
Component	Unit Price (NTD)	Quantity	Subtotal (NTD)
STM32F401CCU6	\$400	1	\$400
GY-521 / MPU6050	\$95	1	\$95
MG90S Servo	\$50	4	\$200
Clipboard	\$29	1	\$29
Pulley	\$19	4	\$76
String	\$14	1	\$14
Universal Joint	\$76	1	\$76
Socket	\$23	1	\$23
Screw Eye	\$12	1 pack	\$12
Serrated Flange Nut	\$20	1 pack	\$20
Washer	\$10	1 pack	\$10
Corrugated Board	\$12	1	\$12
Acrylic Sheet	\$140	1	\$140
Dupont Wires	\$20	lots	\$20
Arduino Mega	(\$400)	1	(\$400)
ST-link V2	(\$200)	1	(\$200)
Total			\$1127(1727)

Quick Demo

Quick Live Demo

(We will have a more detailed demo at the end of the presentation)

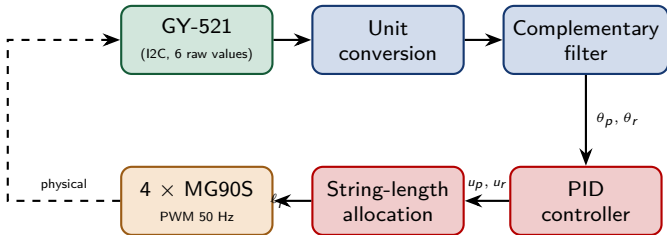
Mechanical Structure



- 4 strings pulling 4 platform corners (PA0 back, PA1 left, PA2 right, PA3 front)
- Cross-pairs work antagonistically: one pulls in, the other releases
- Central universal joint bears the load; servos only adjust angle

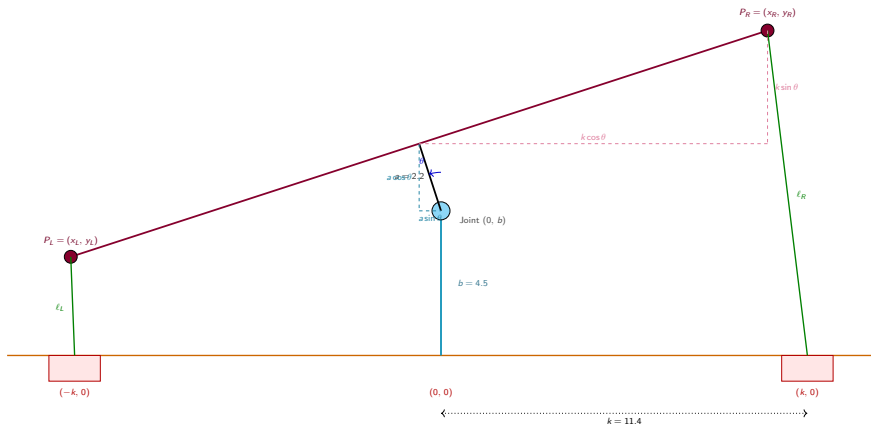
Acknowledgement: Chiin.H().

Signal Flow Diagram



- Closed-loop at **500 Hz** (2 ms control period)
- Pitch and roll axes controlled independently
- All computation runs on STM32F401, USB-CDC for logging

Geometric Setup of the Mechanism



Platform tilts about the universal joint at $(0, b)$. The *upper pillar* (length a) rigidly connects the joint to the platform center, so it tilts with the platform. Strings run from platform ends to fixed servos at $(\pm k, 0)$.

Servo Action

Corrections u_p , u_r in **degrees of attitude correction**.

But each servo controls the **length of a string** that pulls the platform.

Naive approach (does not work well)

Map servo angle directly to angle output: $\theta_{\text{servo}} + = u$.

Problem: 1° of servo rotation does *not* produce 1° of platform tilt. The relationship is highly nonlinear because strings connect a rotating platform to fixed servo positions.

Our approach

Compute the *exact string length* the platform would need at the desired tilt angle, then convert that length into a servo rotation via the spool radius.

Deriving the String Length (Step 1: Locate Platform Center)

The upper pillar rotates rigidly with the platform about the joint at $(0, b)$. The platform *center* is the top of the upper pillar. Before rotation it sits at $(0, b + a)$. The offset from the joint is $(0, a)$. After rotation by θ :

$$\text{offset}' = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} 0 \\ a \end{bmatrix} = \begin{bmatrix} -a \sin \theta \\ a \cos \theta \end{bmatrix}$$

Add back the joint position $(0, b)$:

$$P_{\text{center}} = (-a \sin \theta, b + a \cos \theta)$$

This term is what makes our model different from a naive single-pillar setup. Because a is not negligible compared to k , ignoring it would introduce a position error of up to $a \sin \theta_{\max}$ in the platform center.

Deriving the String Length (Step 2: Platform Endpoints)

Each platform endpoint is offset $\pm k$ along the platform direction. After rotation, the offsets become $(\pm k \cos \theta, \pm k \sin \theta)$.

Add to the platform center P_{center} :

$$P_R = (-a \sin \theta + k \cos \theta, b + a \cos \theta + k \sin \theta)$$

$$P_L = (-a \sin \theta - k \cos \theta, b + a \cos \theta - k \sin \theta)$$

String length is the Euclidean distance from each endpoint to its servo:

$$\ell_R = \|P_R - (k, 0)\|, \quad \ell_L = \|P_L - (-k, 0)\|$$

Result

$$\ell_R(\theta) = \sqrt{(-a \sin \theta + k \cos \theta - k)^2 + (b + a \cos \theta + k \sin \theta)^2}$$

$$\ell_L(\theta) = \sqrt{(-a \sin \theta - k \cos \theta + k)^2 + (b + a \cos \theta - k \sin \theta)^2}$$

Sanity Check at $\theta = 0$

Substituting $\sin 0 = 0$, $\cos 0 = 1$:

$$\ell_R(0) = \sqrt{(0 + k - k)^2 + (b + a + 0)^2} = a + b$$

$$\ell_L(0) = \sqrt{(0 - k + k)^2 + (b + a - 0)^2} = a + b$$

Both strings have length $a + b$ at rest ✓

String length *change* relative to neutral position:

$$\Delta\ell_R(\theta) = \ell_R(\theta) - (a + b), \quad \Delta\ell_L(\theta) = \ell_L(\theta) - (a + b)$$

Numerical example: $a = 2.2$, $b = 4.5$, $k = 11.4$, $\theta = 10^\circ$

$$\ell_R(10^\circ) \approx 8.66 \text{ cm}, \quad \Delta\ell_R \approx +1.96 \text{ cm (release)}$$

$$\ell_L(10^\circ) \approx 4.69 \text{ cm}, \quad \Delta\ell_L \approx -2.01 \text{ cm (reel in)}$$

The two strings move *antagonistically* (one shortens, one lengthens) — exactly the behavior we want.

From String Length to Servo Angle

Each servo has a spool with circumference C . One full 360° revolution winds/unwinds one full circumference of string. Therefore:

$$\Delta\ell = C \cdot \frac{\alpha}{360^\circ} \quad \Rightarrow \quad \boxed{\alpha = \frac{360^\circ \cdot \Delta\ell}{C}}$$

where α is the servo rotation (in degrees) and $\Delta\ell$ is the required string-length change.

Final servo command (relative to the 90° neutral position):

$$\theta_{\text{servo}} = 90^\circ + \alpha$$

Continuing the example ($\Delta\ell_R = +1.96$ cm, $C = 2\pi r$ with $r = 1.0$ cm $\Rightarrow C \approx 6.28$ cm)

$$\alpha_R = \frac{360 \cdot 1.96}{6.28} \approx 112^\circ$$

A 112° servo rotation is required to release 1.96 cm of string — which is why bigger spool circumference gives finer (but smaller-range) control.

What the MPU6050 Measures

Two sensors in one chip

Accelerometer: measures *specific force* (in g)

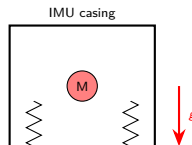
Gyroscope: measures *angular rate* (in deg/s)

Accelerometer (counter-intuitive!)

Even when stationary, the chip reads 1 g on z because:

$$\vec{a}_{\text{meas}} = \vec{a}_{\text{motion}} - \vec{g}$$

The spring inside resists gravity $\Rightarrow$
measurement is the reaction force.



Fuse Two Sensors

	Accelerometer	Gyroscope
Steady state	Accurate (gravity)	Slow drift
Fast motion	Corrupted by inertia	Accurate
Long term	No drift	Drift grows with t
Good in	Low frequency	High frequency

Key Idea

Use accelerometer for low-frequency reference (gravity is reliable)

Use gyroscope for high-frequency dynamics (no motion artefacts)

Blend them with a **frequency-domain crossover** $\Rightarrow$ Complementary Filter

Kalman filter would adapt the weights dynamically, but the fixed-weight complementary filter is sufficient for our low-speed platform and is far cheaper computationally (1 multiply + 1 add per axis).

The Complementary Filter

Discrete recursion

$$\hat{\theta}_k = \alpha \left(\hat{\theta}_{k-1} + \omega_k \Delta t \right) + (1 - \alpha) \theta_k^{\text{acc}}$$

- $\alpha \approx 0.96$: trust gyroscope short-term, use accelerometer to slowly correct drift.
- Equivalent to: **high-pass on gyro + low-pass on accel**, summing to unity gain.

Control Problem Statement

Objective

Drive the platform attitude (θ_p, θ_r) to $(0, 0)$ in the presence of disturbances.

Define the error signal:

$$e(t) = \theta_{\text{ref}}(t) - \theta(t) = 0 - \theta(t) = -\theta(t)$$

We want a control law $u(t)$ (servo correction in degrees) that:

- Reacts quickly to disturbances
- Eliminates steady-state error (when the table itself is tilted)
- Avoids overshoot and oscillation

⇒ **PID controller**, the workhorse of feedback control (Week 11).

The Three Terms of PID

Continuous form

$$u(t) = \underbrace{K_p e(t)}_{\text{Proportional}} + K_i \underbrace{\int_0^t e(\tau) d\tau}_{\text{Integral}} + K_d \underbrace{\frac{de(t)}{dt}}_{\text{Derivative}}$$

Term	Reacts to	Cures	Side effect
P	Current error	Slow response	Steady-state error
I	Accumulated error	Steady-state error	Overshoot, slow
D	Rate of change of error	Overshoot, damping	Noise amplification

Analogy:

P = spring (pulls toward target), I = memory (accumulates over time),

D = damper (resists motion)

Discrete Implementation

Approximate the integral by rectangular sum, derivative by backward difference:

$$\int_0^{t_k} e d\tau \approx \sum_{i=0}^k e_i \Delta t, \quad \frac{de}{dt} \approx \frac{e_k - e_{k-1}}{\Delta t}$$

Discrete PID

$$u_k = K_p e_k + K_i \cdot \Delta t \sum_{i=0}^k e_i + K_d \cdot \frac{e_k - e_{k-1}}{\Delta t}$$

Two practical safeguards:

- **Anti-windup:** clip the integral so it cannot grow unboundedly during saturation
- **Output saturation:** clip u to the servo's physical range

Discrete Velocity Form: PI Implementation

Standard Velocity Form PID: Subtracting positional u_{k-1} from u_k yields the required *change* Δu_k :

$$\Delta u_k = u_k - u_{k-1} = K_p[e_k - e_{k-1}] + K_i \Delta t e_k + \frac{K_d}{\Delta t} [e_k - 2e_{k-1} + e_{k-2}]$$

Our Velocity Form PI Implementation

To avoid noise amplification from the second difference (D term), our firmware omits the K_d term and implements a **Velocity Form PI** controller:

$$\Delta u_k = K_{P_vel} [e_k - e_{k-1}] + K_{I_vel} e_k$$

Engineering Insight: In microcontrollers, the second difference $[e_k - 2e_{k-1} + e_{k-2}]$ is highly sensitive to sensor noise. Dropping it ensures a smoother control signal.

Velocity Form PI: Mathematical Equivalence

Why does it work? (When Accumulated)

Accumulating (integrating) the velocity step Δu_k over time mathematically reconstructs the standard positional control law:

$$u_k = \sum_{j=0}^k \Delta u_j = K_{P_vel} e_k + K_{I_vel} \sum_{j=0}^k e_j$$

- **K_{P_vel} acts as Proportional (P):**

Multiplies the first difference $[e_k - e_{k-1}]$. When accumulated, the intermediate terms cancel out (telescoping sum), leaving just the current error e_k .

- **K_{I_vel} acts as Integral (I):**

Multiplies the current error e_k directly. When summed over time, it naturally becomes the accumulated historical error ($\sum e_j$).

Full Control Pipeline

- ① Read IMU, compute θ_p and θ_r via complementary filter.
- ② Run two independent PIDs:

$$u_p = \text{PID}_p(0 - \theta_p), \quad u_r = \text{PID}_r(0 - \theta_r)$$

- ③ For each axis, compute the required string lengths using the geometric formulas:

$$\ell_R = \ell_R(u_p), \quad \ell_L = \ell_L(u_p)$$

- ④ Convert each string length to a servo rotation:

$$\alpha_i = \frac{360^\circ \cdot (\ell_i - (a + b))}{C}$$

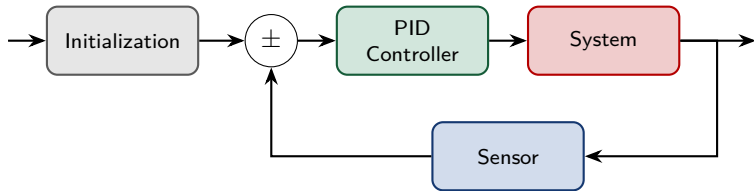
- ⑤ Output PWM via TIM2 channels.

Same procedure runs for the roll axis.

All computations complete within one 2 ms control period.

Implementation

Overview of our closed-loop control system:



Implementation

Code Structure

- **IMU Acquisition (mpu6050.c):**
 - Initialization and calibration
 - Read single frame data
- **Servo Control (servo.c):**
 - Initialization
 - Position-based PWM
- **Attitude Estimation (attitude.c):**
 - Initialization and calibration
 - Complementary filter
- **Control (main.c):**
 - Initialize the servo and IMU modules
 - Core closed-loop control with PI controllers
 - Servo angle calculation

Implementation: Phase 1

Initialization

Three main categories for initializaing:

- **Geometry Bounds:**

- Upper/Lower pillar length and Servo position
- Min/Max tilt angles (to prevent over-rotation and string slack)
- Servo PWM bounds, reversal flags
- Servo soft deadband and step limit (limit max velocity and ignore small errors)

- **Fixed Control Parameters:**

- Loop cycle time
- PID gains K_{P_vel} and K_{I_vel}
- Long-term drift compensation
- Target angle leak gain (to prevent cumulative error)

- **Sensor Calibration:**

- Accelerometer and gyroscope bias (averaging 500 samples at startup)
- Complementary filter $\alpha = 0.96$ (favoring gyro instead of accelerometer for stability)
- Sweep calibration for pitch and roll angles

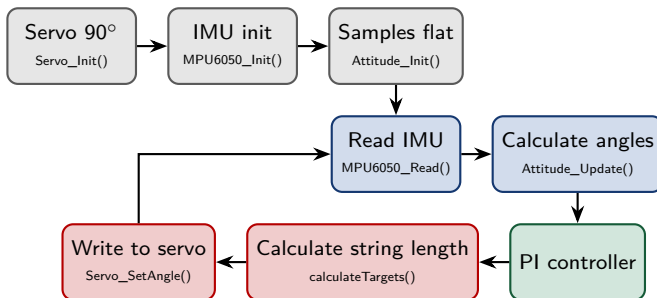
Implementation: Phase 2

Closed Loop Control (Full Control Pipeline)

- 1 IMU Acquisition: read raw data from MPU6050, then convert to physical units.
- 2 Attitude Estimation: compute θ_p and θ_r with complementary filter.
- 3 PI Control Calculation: compute u_p and u_r
- 4 String Length Calculation: compute l_R and l_L
- 5 Servo Angle Calculation: compute α_R and α_L

Implementation

Full Control Pipeline



Task Model

Single-rate non-preemptive sequential execution

Control period $T = D = 2000 \mu\text{s}$ (500 Hz). Tasks run in fixed order:

$\tau_1 \rightarrow \tau_2 \rightarrow \tau_3 \rightarrow \tau_4 \rightarrow \tau_5$.

	Task	Mean (μs)	P99 (μs)	WCET (μs)	Util.
τ_1	IMU Read (I2C)	1558	1558	1573	78.6%
τ_2	Complementary Filter	6	6	7	0.4%
τ_3	PID Control	2	3	3	0.1%
τ_4	Cable Geometry	31	41	42	2.1%
τ_5	Servo PWM Update	0	2	4	0.2%
	Total	1597		1629	81.5%

Measured with STM32 DWT cycle counter; 36 031 samples.

Schedulability Analysis

Exact test (non-preemptive, single-rate) — necessary & sufficient

$$\sum_i C_i \leq D \implies 1573 + 7 + 3 + 42 + 4 = 1629 \leq 2000 \quad \textbf{SCHEDULABLE}$$

$$\text{Slack} = D - \sum C_i = 371 \mu\text{s} \quad (18.6\% \text{ margin})$$

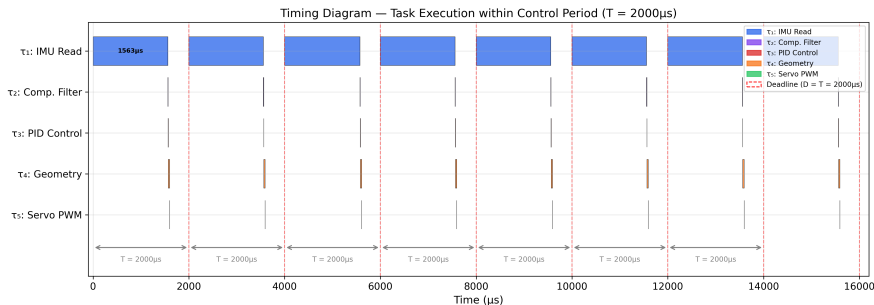
Liu & Layland RM bound vs. our system

The classical RM utilisation bound for $n = 5$ is $U \leq n(2^{1/n} - 1) = 74.3\%$. Our $U = 81.5\%$ exceeds this bound — but that bound applies to *preemptive* RM scheduling. Our system is **non-preemptive single-rate**; the exact test $\sum C_i \leq D$ is the correct criterion.

Empirical verification (36 031 samples)

Deadline misses: **0 / 36 031 (0.00%)** Observed WCET: $1614 \mu\text{s}$ Mean:
 $1597 \mu\text{s}$ P99: $1608 \mu\text{s}$

Timing Diagram — Tasks within the Control Period



Each $T=2000 \mu s$ period runs the five tasks in sequence. τ_1 (IMU read, $\sim 1563 \mu s$) dominates; τ_2 – τ_5 are negligible by comparison. All tasks finish well before the red deadline line $\Rightarrow \sum_i C_i = 1629 \leq D = 2000 \mu s$, zero misses.

Lyapunov's Method — Setup

Lyapunov's direct method (informal)

For equilibrium $\mathbf{x} = \mathbf{0}$, if a scalar $V(\mathbf{x})$ has (i) $V(\mathbf{0})=0$ and $V(\mathbf{x})>0$ for $\mathbf{x}\neq\mathbf{0}$ (positive definite), and (ii) V radially unbounded, then: $\dot{V} \leq 0 \Rightarrow$ **stable / bounded**; $\dot{V} < 0 \Rightarrow$ **asymptotically stable**.

State: $\mathbf{x} = [\theta_p, \theta_r]^\top$. **Equilibrium:** $\theta_p = \theta_r = 0$ (platform level).

Lyapunov candidate — quadratic “tilt” function

$$V(\theta_p, \theta_r) = \frac{1}{2}\theta_p^2 + \frac{1}{2}\theta_r^2 = \frac{1}{2}\|\mathbf{x}\|^2$$

Squared distance from level: $V=0$ when flat, larger when tilted. It is a *distance metric* (analogous to spring energy $\frac{1}{2}kx^2$), not literal mechanical energy.

Continuous-Time Derivative of V

Differentiate by the chain rule ($\frac{d}{dt}\theta^2 = 2\theta\dot{\theta}$):

$$\dot{V} = \frac{d}{dt}\left(\frac{1}{2}\theta_p^2 + \frac{1}{2}\theta_r^2\right) = \underbrace{\frac{1}{2} \cdot 2\theta_p}_{\text{outer}} \underbrace{\dot{\theta}_p}_{\text{inner}} + \frac{1}{2} \cdot 2\theta_r \dot{\theta}_r = \boxed{\theta_p \omega_p + \theta_r \omega_r}$$

where $\omega = \dot{\theta}$ is the angular rate (gyro: gy $\rightarrow$ pitch, gx $\rightarrow$ roll).

Sign tells us "better or worse" right now: tilted & returning (θ, ω opposite sign) $\Rightarrow \dot{V} < 0$ (improving); tilted & overshooting (same sign) $\Rightarrow \dot{V} > 0$ (worsening).

Why $\dot{V}$ cannot be signed analytically

Our controller is **velocity-form PI** — the K_D term was dropped, so there is *no explicit rate-damping term*; and we have no identified plant model relating ω to θ . Hence $\dot{V} = \theta \cdot \omega$ is **not sign-definite** on paper (it is > 0 during every overshoot). $\Rightarrow$ we verify the decrease condition **quantitatively from data**, via two measures: the decay rate γ and the $\dot{V} \leq 0$ fraction.

The $\dot{V} \leq 0$ Fraction — What It Measures & Why

We also count, sample by sample, how often the energy is *decreasing*:

$$\rho_{\Delta V} = \frac{1}{M-1} \sum_{k=0}^{M-2} \mathbf{1}[V[k+1] - V[k] \leq 0] \times 100\%$$

where $\mathbf{1}[\cdot]$ is the indicator (1 if true, 0 otherwise). In code: `dV = np.diff(V); frac = np.mean(dV <= 0)*100`.

Why this equals the Lyapunov decrease test

Since $\dot{V} \approx \frac{V[k+1] - V[k]}{\Delta t}$ and $\Delta t > 0$, $\text{sign}(\Delta V) = \text{sign}(\dot{V})$. So $\rho_{\Delta V}$ is the fraction of time the Lyapunov condition $\dot{V} \leq 0$ holds. We check only the *sign* (not $/\Delta t$), i.e. the **direction** of energy change, not its speed.

Interpreting $\rho_{\Delta V} \approx 50\%$ (our data)

An **under-damped** system oscillates: energy *rises* during overshoot ($\sim$ half the time) and *falls* during return ($\sim$ half). So $\rho_{\Delta V} \approx 50\%$ is the **signature of oscillation, not instability** — like a swing whose height goes up-down each pass but whose *peaks* shrink. Convergence is judged by the envelope ($\gamma > 0$), not by the per-sample count.

Decay Rate γ — What It Is and Where It Comes From

Beyond $\dot{V} < 0$, the *quantitative* stability condition is that energy decays **proportionally to itself**:

$$\dot{V} \leq -\gamma V, \quad \gamma > 0.$$

Solving the equality case (a 1st-order linear ODE):

$$\frac{dV}{dt} = -\gamma V \Rightarrow \frac{dV}{V} = -\gamma dt \xRightarrow{\int} \ln V = -\gamma t + C \Rightarrow \boxed{V(t) = V_0 e^{-\gamma t}}$$

This is **exponential decay** — the same form as a damped spring, an RC discharge, or Newtonian cooling. (V_0 = energy at disturbance onset.)

How we extract γ from the log

Take logs: $\ln V(t) = \ln V_0 - \gamma t$ — a straight line. We fit the *peak envelope* of V after each disturbance and read the slope:

$$\gamma = -\text{slope of } \ln V_{\text{peak}} \text{ vs } t, \quad \tau = 1/\gamma \text{ (time constant).}$$

$\gamma > 0 \Rightarrow V$ decays (converging); $\gamma = 0 \Rightarrow$ no decay; $\gamma < 0 \Rightarrow V$ grows (unstable). **Our result:** $\gamma = 0.275 \text{ s}^{-1}$ ($\tau \approx 3.6 \text{ s}$).

Quantitative Stability — Results & Verdict

From 36,031 cycles (80 s), 4 disturbance events:

Event	V_{peak}	$\gamma \text{ (s}^{-1}\text{)}$	$\tau \text{ (s)}$	$\rho_{\Delta V}$
#1	22.45	0.158	6.3	49.1%
#2	45.29	0.317	3.2	49.7%
#3	42.11	0.304	3.3	48.9%
#4	27.62	0.320	3.1	51.0%
avg	—	0.275	3.6	49.7%

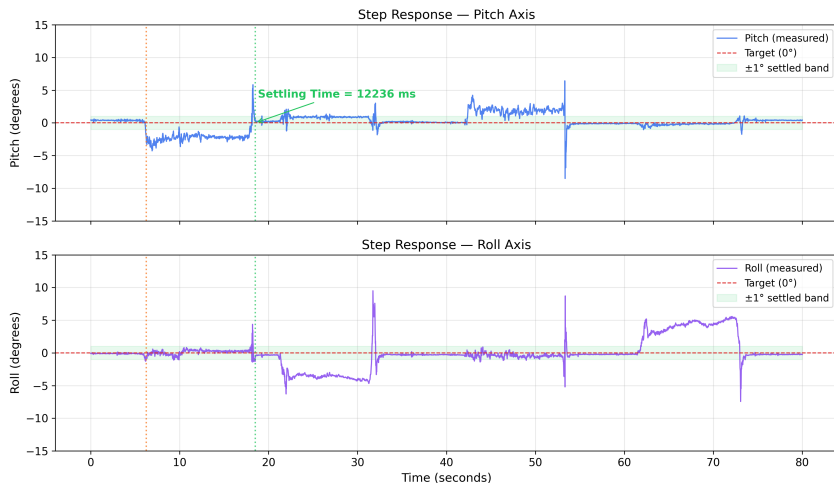
Verdict

Bounded + decaying envelope + converges to a $\sim 3.4^\circ$ ball $\Rightarrow$ **practically stable / ultimately bounded**. *Not* asymptotic to 0.

Three criteria, all met:

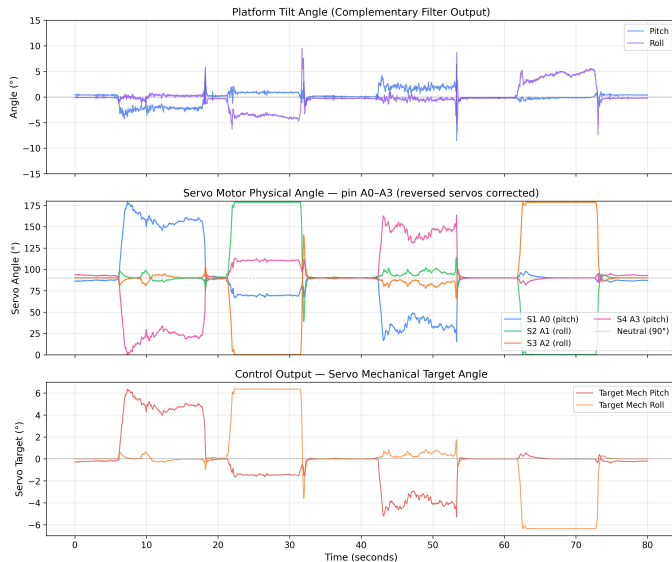
- 1 **Bounded:** $V_{\text{max}} = 45.3 < \infty$ (no escape).
- 2 **Convergent:** envelope decays, $\gamma = 0.275 \text{ s}^{-1}$.

Step Response — Pitch & Roll Axes



Step response (4 disturbance events). Average settling time ≈ 11.9 s (to $\pm 1^\circ$ band). Peak disturbance $\approx 8.2^\circ$. Platform recovers consistently.

Control Signal & Servo Behaviour



Performance Summary

Real-Time Performance

- Control frequency: **500 Hz**
($T=2000\ \mu\text{s}$)
- WCET: $1629\ \mu\text{s}$ ($< 2000\ \mu\text{s}$ ✓)
- CPU utilisation: 81.5%
- Deadline margin: 18.6%
- Deadline miss rate: **0.00%**

Control & Stability

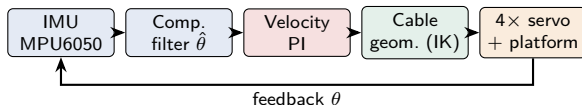
- Avg. settling time: 11.96 s
- Steady-state RMS: 3.38°
- Lyapunov V : bounded
($V_{\max}=45.3$)
- Decay rate: $\gamma=0.275\ \text{s}^{-1}$ ✓
- Stability: **practical / ultimately bounded**

Main bottleneck: IMU I2C (78.6% of budget)

Upgrading to SPI ($\sim 400\ \mu\text{s}$) would cut τ_1 WCET by $\sim 75\%$, freeing $\sim 1600\ \mu\text{s}$ of slack for a faster loop or a true damping term.

Project Summary — The Complete CPS Pipeline

One closed loop: **sense** → **estimate** → **control** → **actuate** → **move** → **sense**.



Each stage

- Sense/estimate: gyro+accel → drift-free $\hat{\theta}$.
- Control: velocity-form PI (u = integrator) + leak.
- Actuate: IK maps tilt → cable → servo angle.

Outcomes

- Real-time: 500 Hz; $1629 \leq 2000 \mu\text{s}$; **0** misses.
- Stability: V bounded, $\gamma=0.275 \text{ s}^{-1}$; ball $\sim 3.4^\circ$.
- **Practically stable / ultimately bounded.**

Conclusion & Future Work

What we built and verified:

- A complete cyber-physical loop: IMU $\rightarrow$ complementary filter $\rightarrow$ velocity-form PI $\rightarrow$ inverse kinematics $\rightarrow$ PWM $\rightarrow$ platform.
- **Formal schedulability:** $\sum C_i = 1629 \leq D = 2000 \mu s$, 0 deadline misses across 36,031 cycles.
- **Quantitative stability:** Lyapunov V bounded & decaying ($\gamma=0.275$) $\Rightarrow$ practically stable / ultimately bounded.

Future work (motivated directly by the analysis):

- **IMU to SPI** ($\rightarrow$ free $\sim 1200 \mu s$) for a faster loop.
- Systematic K_P, K_I tuning; Kalman filter for attitude.

Thank you!

References



STMicroelectronics.

RM0383 Reference manual: STM32F411xC/E advanced Arm®-based 32-bit MCUs.

Rev. 3, STMicroelectronics.



InvenSense Inc.

MPU-6050 Product Specification.

Rev. 3.4, InvenSense.